

Bonus: More Experiments

These exercise ideas are exploratory and don't follow a specific order. Delve into the ones that pique your interest. Research and learn about each of the topics you choose.

Work on the following exercises using your processed **house pricing dataset**.

The star ★ indicates the ones considered more relevant based on your level.

★ Error Distributions

- Display a scatter plot to compare predicted values (x-axis) against real ones (y-axis). Are there specific value ranges where your model consistently under-performs?
 - You can use the function `cross_val_predict` from *sklearn* to obtain predictions for your entire training set using cross-validation. This function trains the model on each training fold and returns the predictions made on the corresponding validation fold, so every prediction comes from data the model **has not seen during training**.

★ Metrics

- Familiarize yourself with various evaluation metrics and assess how your top-performing models measure against them:
 - Median Absolute Error (MedAE).
 - Root Mean Squared Error (RMSE).
 - R-squared (R²).
 - Mean Absolute Percentage Error (MAPE).
 - This metric has an important problem... which one?
 - Symmetric MAPE (SMAPE) fixes it.

Different K for Cross-validation

- Examine the impact of using various K values in cross-validation.
Consider that the optimal K value isn't solely determined by the best performance! Why?
 - Visualize performance metrics of a chosen model with different K values for cross-validation splits. Depict both training and validation scores.
 - Chart the computation time associated with different K values for cross-validation.

Learning Curves

- Conduct experiments using one or more of your top models, training them with different portions of data.
 - For example, split your training set so that 10% of the data is used for training and the remaining 90% for validation.
You don't need to use cross-validation here, simply fit the model on the small training portion and evaluate its performance on the larger validation set.
 - Analyze the outcomes and visualize the results for varying data portions like 10–90, 20–80, and so on.

★ Dimensionality Reduction

- Learn about Principal Component Analysis (PCA). There is no need to understand all the details.
 - Display a scatter plot of your data when using only 2 principal components.
 - Plot a chart depicting model performance (y-axis) by altering the number of principal components (x-axis).
 - Draw conclusions from your findings.
- If RF has been explained in class: elect the most important features (e.g., based on RF feature importances) and retrain your top models. Observe any changes in performance.
- Understand Feature Recursive Elimination (RFE) and apply it. Note any performance improvements or declines.
- Learn about and explore t-SNE. If computation time becomes an issue, reduce your dataset's size. Gauge if this changes model performance.
- Likewise, experiment with UMAP.

★ Linear Models

- Delve deeper into Linear Regression by exploring ISL section 3.1 (pages 70 to 80):
https://drive.google.com/file/d/1ajFkHO6zjrdGNqhQW1jKBZdiNGh_8YQ1/view
- Learn about regularized linear models: Ridge, Lasso, and ElasticNet. Train them on your dataset and contrast their performance against other models.
 - “Introduction to Statistical Learning” and “Hands-on ML sklearn, keras & TF” books have good explanations about these models.
 - What are the differences in performance and training time with and without feature scaling?

★ Outliers

- Remove outliers from your target variable. Do your model's performance improve?
- Learn about the effect of transforming your target variable before training, and reverting it when predicting. For instance, a logarithmic transformation for training and using the exponential function for predictions. Consider using the `TransformedTargetRegressor` in sklearn.

Feature selection

- Apply a feature selection technique to select the features that are more important for prediction. For example:
 - RFE (check `RFE` in sklearn).
 - Most N important features based on RF or GBDT (only if they have been covered in class).
- After training the models with the most important features, do they improve performance?

Tree-based Models

(Only proceed if trees have been covered in class)

- Understand the concept of the Out-of-bag (OOB) estimate. Check in in your RF model and compare the results with validation scores.
- Learn about Extratrees and apply it.
- Try other libraries better suited for tree-based models than sklearn, e.g., XGBoost or LightGBM.

Bagging

(Only proceed if bagging has been discussed in class)

- Use Bagging with ML algorithms other than Decision Trees, such as LR or kNN. Check `BaggingRegressor` in sklearn. Interpret the outcomes and theorize why certain results occurred. Consider ways to enhance performance.

Boosting

(Only proceed if boosting has been explained in class)

- Employ Gradient Boosting with algorithms apart from Decision Trees, like LR or kNN. Check `GradientBoostingRegressor` in sklearn. Interpret the outcomes and theorize why certain results occurred. Consider ways to enhance performance.
- Learn about AdaBoost and apply it.

★ Other Ensembling Techniques

- Learn about stacking and apply it.
- Learn about blending and apply it.
- Learn about cascading and apply it.

Interpretability

- Learn about Permutation Feature Importance and apply it to understand the contribution of each feature to the prediction. Check `permutation_importance` in sklearn.
- Likewise, learn and experiment with SHAP (SHapley Additive exPlanations).
- Likewise, learn and experiment with LIME .

Hyperparameter Optimization

(Only proceed if hyperparameter optimization has been introduced in class)

- Learn about Bayesian optimization and apply it.

More Features

- Derive (from the existing attributes) more features that can be useful for prediction.
- Add features from external data (e.g., Related to Ames (Iowa) in 2008-2010) that can be helpful for prediction.

Automatic ML

- Explore automated machine learning using libraries such as Auto-sklearn, H2O AutoML, Optuna, TPOT, among others.

Full Pipeline

- Envision an automated system for regression that periodically receives new data. Design a comprehensive pipeline to emulate this process: data preprocessing, train/test splitting, hyperparameter tuning, validation, testing, and monitoring (including performance evaluation logs and potential failure alerts).
- Model Drift Detection: Implement mechanisms to detect if the model's performance degrades over time as new data becomes available.